



Handbuch

Deine Website, verständlich gemacht.

Wie sie funktioniert. Wie du sie selbst pflegst.

Für Rolf Schnee
Kusaidia Afrika — Helfen in Afrika e.V.
Mai 2026



BEVOR DU ANFÄNGST

Vorwort

Lieber Rolf,

dieses Heft ist für dich gedacht. Es soll dir helfen, deine neue Website nicht nur zu benutzen, sondern auch zu **verstehen** — und kleine Änderungen selbst vorzunehmen.

Du brauchst dafür keinerlei Vorkenntnisse. Wir fangen vorne an: Was ist eigentlich eine Website? Wie ist sie aufgebaut? Wo steht der Text, den ich auf der Startseite sehe? Wie ändere ich ein Datum oder ein Foto?

Was du am Ende kannst

- Texte auf der Website ändern.
- Eine neue Aktuelles-Meldung schreiben.
- Bilder austauschen oder neue hinzufügen.
- Eine ganz neue Seite anlegen.
- Erkennen, was du im Code siehst — auch wenn du nicht jedes Detail verstehst.
- Mit Git deinen Arbeitsstand sichern und wieder zurückrollen.

Lass dir Zeit. Lies in der Reihenfolge, in der das Heft aufgebaut ist. Springe zurück, wo es nicht ganz hängenbleibt. Und ruf an, wenn etwas klemmt.

*Herzlich,
Thomas*

WAS DICH ERWARTET

Inhalt

Teil 1 · Was ist eigentlich eine Website?

- Eine Website ist nur eine Sammlung von Dateien
- Drei „Sprachen“ arbeiten zusammen
- Was passiert beim Aufrufen?

Teil 2 · Die Werkzeuge

- Der Browser (kennst du schon)
- Der Editor — VS Code
- Die Ordnerstruktur deiner Website

Teil 3 · HTML verstehen

- Tags sind wie Klammern
- Die wichtigsten Tags
- Attribute — die Eigenschaften eines Tags
- Die Struktur einer ganzen Seite
- Walkthrough: deine Startseite

Teil 4 · CSS auf einen Blick

- Was CSS macht
- Wo deine Farben definiert sind
- Wann musst du CSS anfassen?

Teil 5 · Häufige Aufgaben

- A · Einen Text ändern
- B · Ein Bild austauschen
- C · Eine neue Aktuelles-Meldung
- D · Eine ganz neue Seite anlegen
- E · Einen Link einfügen
- F · Einen Spendenbetrag anpassen
- G · Ein Vorstandsmitglied ändern
- H · Ein Bild zur Galerie hinzufügen

Teil 6 · Git — der Schutzschirm

- Was ist Git?
- Die drei wichtigsten Befehle
- Wenn etwas kaputt geht

Teil 7 · Wenn etwas nicht funktioniert

- Häufige Fehler & Lösungen
- Die Browser-Konsole

Anhang · Nachschlagen

- A · Glossar

- B · Spickzettel der Tags
- C · Dateibaum deiner Website

DAS FUNDAMENT — DREI SEITEN

Teil 1 Was ist eigentlich eine Website?

Bevor wir Code anschauen, klären wir das große Bild. Eine Website ist nicht magisch, sie ist nicht „im Internet“ auf besondere Weise — sie besteht aus ganz normalen Dateien.

Kapitel 1 · Eine Website ist nur eine Sammlung von Dateien

Auf einem Computer irgendwo auf der Welt — wir nennen ihn **Server** — liegen deine Dateien. Genau wie auf deinem eigenen PC, wo du auch Dokumente, Fotos und Tabellen hast. Nur dass dieser Server immer angeschaltet ist und immer im Internet erreichbar bleibt.

Wenn jemand `kusaidia-afrika.de` in den Browser tippt, fragt der Browser diesen Server: „Gib mir die Startseite!“ — und der Server schickt eine Datei zurück. Der Browser baut daraus die Seite zusammen, die du siehst.

Deine ganze Website besteht aus rund 30 solchen Dateien. Sie liegen in einem Ordner auf deinem PC unter:

```
C:\Users\rolf\Documents\GitHub\Kusaidia\
```

Wenn du die Website veröffentlichst (das macht dein Enkel oder ein Hosting-Dienst), werden genau diese Dateien auf den Server kopiert. Mehr passiert dabei nicht.

Kapitel 2 · Drei „Sprachen“ arbeiten zusammen

Jede Website spricht **drei Sprachen** gleichzeitig. Jede hat ihre Aufgabe:

HTML	der Inhalt	Überschriften, Absätze, Bilder, Links. Was auf der Seite <i>steht</i> .
CSS	das Aussehen	Farben, Schriften, Abstände, Größen. Wie es <i>aussieht</i> .
JavaScript	das Verhalten	Diashow, Menü auf-/zuklappen, Klick-Reaktionen. Was sich <i>bewegt</i> .

Eine Analogie: Stell dir ein Haus vor.

- **HTML** sind die Mauern, Wände, Türen. Sie bilden die Struktur.
- **CSS** ist die Farbe der Wände, der Bodenbelag, die Tapete.
- **JavaScript** sind die Lichtschalter, die Türklingel, das, was reagiert wenn du es berührst.

Du wirst hauptsächlich mit **HTML** in Kontakt kommen. CSS schaust du dir nur kurz an, JavaScript brauchst du gar nicht zu verstehen — es funktioniert von selbst.

Kapitel 3 · Was passiert beim Aufrufen?

Wenn jemand deine Website besucht, läuft folgendes ab — und zwar in Sekundenbruchteilen:

1. Browser fragt den Server: „Gib mir die Datei *index.html*!“
2. Server schickt sie.
3. Browser liest die Datei und merkt: „Aha, ich brauche auch das Stylesheet *style.css* und ein paar Bilder.“
4. Browser holt sich auch diese Dateien.
5. Browser baut alles zusammen und stellt die Seite dar.

Wichtig: Wenn du eine Datei änderst, kannst du sofort im Browser F5 drücken, um die neue Version zu sehen. Drücke **Strg + F5**, falls die alte Version noch hängt — das lädt alles komplett neu.

WAS DU BRAUCHST, UM ANZUFANGEN

Teil 2 Die Werkzeuge

Du brauchst genau drei Werkzeuge: einen Browser (hast du schon), einen Editor (installieren wir gleich), und den Datei-Explorer (gibt's bei Windows immer).

Kapitel 4 · Der Browser

Du hast bestimmt schon einen Browser: Chrome, Firefox, Edge — alle funktionieren gleich gut. Damit schaust du dir an, wie deine Website aktuell aussieht.

So öffnest du deine Website lokal:

1. Datei-Explorer öffnen.
2. Zum Kusaidia-Ordner gehen.
3. Doppelklick auf `index.html`.
4. Die Startseite öffnet sich im Browser.

Kapitel 5 · Der Editor — VS Code

Du brauchst ein Programm, mit dem du die HTML-Dateien öffnen und bearbeiten kannst. Theoretisch geht das mit Word oder Notepad — aber das ist mühsam. Es gibt einen kostenlosen, professionellen Editor namens **Visual Studio Code** (kurz: VS Code).

Installation

1. Im Browser `code.visualstudio.com` aufrufen.
2. Auf den blauen Download-Knopf klicken (Windows).
3. Die heruntergeladene Datei öffnen und installieren — Standard-Klicks reichen.
4. VS Code starten.

Den Kusaidia-Ordner öffnen

1. In VS Code: Menü **Datei** → **Ordner öffnen**.
2. Den Kusaidia-Ordner auswählen.
3. Links siehst du nun den Datei-Baum.
4. Klick auf eine Datei (z. B. `index.html`) öffnet sie zum Bearbeiten.

Tipp: VS Code zeigt dir HTML in **bunten Farben** — Tags blau, Attribute orange, Text grau. Das ist *Syntax-Highlighting* und hilft sehr beim Lesen. Du musst nichts dafür tun, es passiert automatisch.

Kapitel 6 · Die Ordnerstruktur deiner Website

Wenn du den Kusaidia-Ordner geöffnet hast, siehst du folgende Struktur:

```
Kusaidia/  
■■■ index.html ← Startseite  
■■■ ueber-uns.html ← Über uns  
■■■ projekte.html ← Übersicht Projekte  
■■■ partner.html ← Partner in Afrika  
■■■ aktuelles.html ← Neuigkeiten  
■■■ unterstuetzen.html ← Spenden & Mitgliedschaft  
■■■ kontakt.html ← Kontakt  
■■■ impressum.html  
■■■ datenschutz.html  
■■■ bildergalerie.html ← Galerie  
■■■ ihre-spende-kommt-an.html ← Transparenz  
■  
■■■ projekte/ ← Projekt-Detailseiten  
■ ■■■ laborantenkolleg-bashanet.html  
■ ■■■ clinical-medicine.html  
■ ■■■ sanu-tec.html  
■ ■■■ madunga-girls-school.html  
■ ■■■ student-support.html  
■  
■■■ en/ ← Englische Version (gleiche Struktur)  
■ ■■■ index.html  
■ ■■■ about.html  
■ ■■■ ...  
■  
■■■ css/  
■ ■■■ style.css ← Aussehen (Farben, Schriften)  
■  
■■■ js/  
■ ■■■ main.js ← Bewegung (Diashow, Menü)  
■  
■■■ assets/  
■■■ images/ ← Alle Fotos  
■■■ hero-1.jpg  
■■■ bischof-antony-lagwen.jpg  
■■■ ...
```

Merke dir vor allem dies: Wenn du Text änderst → es ist eine **.html**-Datei. Wenn du ein neues Bild reinlegst → in den Ordner **assets/images/**. Wenn du eine Farbe ändern willst → in **css/style.css**.

DIE SPRACHE DEINER INHALTE

Teil 3 HTML verstehen

HTML ist eigentlich ganz einfach. Du musst dir nur ein Prinzip merken — und dann die wichtigsten zehn Tags. Das war's.

Kapitel 7 · Tags sind wie Klammern

HTML besteht aus **Tags**. Ein Tag ist wie eine Klammer um einen Text, die dem Browser sagt, was darin steht — eine Überschrift, ein Absatz, ein Bild, ein Link.

Hier das einfachste Beispiel:

```
<p>Hallo, das ist ein Absatz.</p>
```

Was siehst du?

- `<p>` ist die **öffnende Klammer** (p steht für „paragraph“, also Absatz).
- `</p>` ist die **schließende Klammer** (mit Schrägstrich).
- Dazwischen steht der eigentliche Text.

Im Browser wird daraus einfach: *Hallo, das ist ein Absatz.*

Tags kommen **fast immer paarweise**: ein öffnender und ein schließender. Ausnahme: `<img>` für Bilder und ein paar andere — die brauchen keine schließende Klammer, weil sie keinen Inhalt umfassen, sondern *selbst* der Inhalt sind.

Kapitel 8 · Die wichtigsten Tags

Diese zehn musst du kennen — den Rest wirst du im Vorbeigehen lernen:

Tag	Wofür	Beispiel
<code><h1></code> bis <code><h6></code>	Überschriften (h1 = ganz groß, h6 = klein)	<code><h1>Über uns</h1></code>
<code><p></code>	Absatz	<code><p>Ein Absatz Text.</p></code>
<code></code>	Fett gedruckter Text	<code>wichtig</code>
<code></code>	Kursiv (betont)	<code>betont</code>
<code><a></code>	Ein Link	<code>Kontakt</code>

<code></code>	Ein Bild (kein schließender Tag!)	<code></code>
<code></code> und <code></code>	Aufzählung (ul = Liste, li = Eintrag)	<code>EinsZwei</code>
<code>
</code>	Zeilenumbruch	Erste Zeile <code>
</code> Zweite Zeile
<code><div></code>	Allgemeiner Container (ein „Kästchen“)	<code><div>...Inhalt...</div></code>
<code><section></code>	Ein Abschnitt der Seite	<code><section>...</section></code>

Kapitel 9 · Attribute — die Eigenschaften eines Tags

Manche Tags brauchen zusätzliche Informationen. Diese stecken in **Attributen**, die im öffnenden Tag stehen.

Beispiel:

```
<a href="kontakt.html">Schreiben Sie uns</a>
```

Hier ist `href="kontakt.html"` ein Attribut. Es sagt dem Tag `<a>`: „Wenn jemand draufklickt, gehe zur Datei `kontakt.html`.“

Die wichtigsten Attribute

- **href="..."** — wohin der Link führt (beim Tag `<a>`).
- **src="..."** — wo das Bild liegt (beim Tag `<img>`).
- **alt="..."** — eine Beschreibung des Bildes (wichtig für Sehbehinderte und Suchmaschinen).
- **class="..."** — eine „Schublade“ für das Aussehen. Das CSS nutzt sie, um zu bestimmen wie das Element gestaltet wird.
- **id="..."** — ein einzigartiger Name. Brauchst du selten.

Attribute schreibt man immer in der Form **Name="Wert"**. Der Wert steht in Anführungszeichen. Mehrere Attribute werden durch Leerzeichen getrennt.

Kapitel 10 · Die Struktur einer ganzen Seite

Jede deiner HTML-Dateien hat die gleiche Grundstruktur. Es lohnt sich, einmal zu wissen, was wo steht:

```
<!DOCTYPE html>
<html lang="de">
<head>
<!-- Hier stehen Infos für den Browser. -->
<!-- Diese Sachen sieht der Besucher NICHT direkt. -->
<title>Seitentitel im Browser-Tab</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<!-- Alles was du auf der Seite SIEHST, steht hier drin. -->

<header>Navigation oben</header>

<main>
Hauptinhalt – Überschriften, Absätze, Bilder, ...
</main>

<footer>Fußzeile mit Adresse, Links</footer>

</body>
</html>
```

Erklärung Stück für Stück:

- `<!DOCTYPE html>` — sagt dem Browser: „Das ist HTML.“
- `<html lang="de">` — die Wurzel des Dokuments, Sprache ist Deutsch.
- `<head>` — Kopf der Datei. Hier stehen **unsichtbare** Infos wie Seitentitel, Beschreibung und Verweise auf Stylesheet.
- `<body>` — Körper. Hier steht alles, was der Besucher **sieht**.
- `<!-- ... -->` — ein Kommentar. Der Browser ignoriert ihn. Du kannst sowas zur eigenen Erinnerung schreiben.

Kapitel 11 · Walkthrough — deine Startseite

Schauen wir mal kurz in `index.html` hinein. Du brauchst nicht alles zu verstehen — es geht nur ums Wiedererkennen.

Wenn du die Datei in VS Code öffnest und ganz nach oben scrollst, siehst du:

```
<!DOCTYPE html>
<html lang="de">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>Kusaidia Afrika - Helfen in Afrika e.V.</title>
...
</head>
```

Das ist der **Kopf**. Der Titel im Browser-Tab kommt aus dem `<title>`-Tag.

Etwas tiefer beginnt der sichtbare Inhalt:

```
<body>
<header class="site-header">
<div class="site-header__inner">
<a href="index.html" class="brand">
<span class="brand__mark">Kusaidia Afrika</span>
<span class="brand__tag">e.V. seit 2001</span>
</a>
...
</div>
</header>
```

Das ist der Header — die Leiste oben. „Kusaidia Afrika“ und „e.V. seit 2001“ siehst du am oberen Rand der Seite.

Noch weiter unten:

```
<section class="hero">
...
<h1>Kusaidia.<br /><em>Suaheli für »Helfen«.</em></h1>
<p class="hero__lead">
Wir lindern die Not im Gesundheits- und Schulwesen ...
</p>
...
</section>
```

Das ist der Hero — also der große Eindruck oben mit Bild und Überschrift. Hier kannst du Schlagzeile und Lead-Text ändern, indem du den Text zwischen den Tags ersetzt.

Faustregel: Was zwischen den Tags steht, ist der Inhalt, den du gefahrlos ändern kannst. Was *im* Tag steht (z. B. `class="..."`) ist für das Aussehen wichtig — lass das im Zweifel stehen.

DU BRAUCHST NUR WENIG ZU WISSEN

Teil 4 CSS auf einen Blick

CSS bestimmt das Aussehen. Du kommst nur selten damit in Berührung — aber es ist gut zu wissen, wo deine Farben stehen, falls du sie mal ändern willst.

Kapitel 12 · Was CSS macht

CSS heißt *Cascading Style Sheets*. Es bestimmt nicht, **was** auf der Seite ist (das macht HTML), sondern **wie** es aussieht.

Ein CSS-Schnipsel sieht so aus:

```
.btn--primary {  
  background: #b53c2a;  
  color: white;  
  padding: 10px 20px;  
}
```

Übersetzt:

- **.btn--primary** — „alle Elemente mit der Klasse *btn--primary*“.
- **background: #b53c2a;** — „bekommen einen terracotta-roten Hintergrund“.
- **color: white;** — „und weißen Text“.
- **padding: 10px 20px;** — „und drumherum 10px oben/unten, 20px links/rechts Platz“.

Die Datei mit allen CSS-Regeln deiner Website heißt `css/style.css`. Sie hat etwa 900 Zeilen — das musst du alles nicht lesen!

Kapitel 13 · Wo deine Farben definiert sind

Ganz am Anfang von `css/style.css` stehen die **Markenfarben**. Sie heißen „CSS-Variablen“ und werden im Rest der Datei wieder verwendet:

```
:root {  
  --terracotta: #b53c2a; /* Hauptfarbe rot-braun */  
  --saffron: #f8af52; /* Sekundär orange */  
  --vanilla: #fae3a6; /* Heller Akzent */  
  --cream: #fbf7f1; /* Hintergrund warm-weiß */  
  --ink: #1f1a16; /* Textfarbe dunkel */  
}
```

Möchtest du z. B. die Hauptfarbe leicht ändern? Du musst nur diese eine Zeile anfassen. **Alles andere zieht automatisch nach** — Buttons, Überschriften, Akzente. Das ist die Magie von CSS-Variablen.

Achtung: Eine Farbe wird als **#rrggbb** geschrieben — sechs Zeichen aus Ziffern (0–9) und Buchstaben (a–f). Du kannst auf htmlcolorcodes.com Farben suchen und den Code abschreiben.

Kapitel 14 · Wann musst du CSS anfassen?

Selten. Wenn du Text ergänzt, ein Foto austauschst oder eine Aktuelles-Meldung schreibst, kommst du nie mit CSS in Kontakt.

Du musst nur in `style.css` rein, wenn du:

- Eine Farbe ändern willst (siehe Kapitel 13).
- Etwas verschieben/anders anordnen willst.
- Etwas nicht gut aussieht und du es justieren möchtest.

Wenn du nicht weißt, was eine Zeile in `style.css` tut: lass sie stehen. Du kannst nichts dauerhaft kaputtmachen, solange du Git benutzt (Teil 6). Das ist dein Sicherheitsnetz.

HIER WIRD'S PRAKTISCH — SCHRITT FÜR SCHRITT

Teil 5 Häufige Aufgaben

Acht typische Situationen, mit konkreten Klicks. Halte dich an die Reihenfolge — Schritt für Schritt. Speichere zwischendurch oft (Strg + S).

Aufgabe A · Einen Text ändern

Beispiel: Die Telefonnummer hat sich geändert.

1. VS Code öffnen, dann den Kusaidia-Ordner (Datei → Ordner öffnen).
2. Drücke **Strg + Shift + F** — das öffnet die Suche über alle Dateien.
3. Tippe die alte Telefonnummer ein, z. B. 07142 32370.
4. VS Code zeigt alle Stellen, wo sie steht.
5. Klicke das Pfeil-nach-unten-Symbol → es erscheint ein zweites Feld zum Ersetzen.
6. Tippe die neue Nummer ein.
7. Klick auf das Symbol „Alle ersetzen“ (zwei Pfeile in Doppelkreis).
8. Speichere alle Dateien: **Strg + K, dann S** (kurz hintereinander).
9. Im Browser `index.html` öffnen, mit F5 neu laden.

Strg+F = in *dieser* Datei suchen. **Strg+Shift+F** = in *allen* Dateien suchen. Die zweite Form ist oft die mächtigere.

Aufgabe B · Ein Bild austauschen

Beispiel: Du hast ein aktuelleres Bild von Sr. Basilisa Panga.

1. Speichere das neue Bild auf deinem PC. Bedingungen:
 - Format: **.jpg** (oder .png).
 - Größe: max. ca. 2 MB (sonst lädt die Seite langsam).
 - Name: **nur Kleinbuchstaben**, Bindestriche, keine Leerzeichen, keine Umlaute. Beispiel: `sr-basilisa-2025.jpg`.
1. Lege das Bild in den Ordner `assets/images/`.
2. Öffne VS Code. Drücke **Strg + Shift + F**.
3. Suche nach dem alten Dateinamen: `sr-basilisa-panga.jpg`.
4. Ersetze ihn durch deinen neuen Namen.
5. Speichern, im Browser prüfen.

Wenn das Bild nicht erscheint: Schreibfehler im Namen? Liegt es wirklich in `assets/images/`?
Schaue im Browser (F12 → Konsole) nach roten Fehlern.

Aufgabe C · Eine neue Aktuelles-Meldung

Beispiel: Du willst über die Visitationsreise im Herbst 2026 berichten.

1. Öffne `aktuelles.html` in VS Code.
2. Suche den Block `<article>` der Kunstauktion 2024.
3. Markiere ihn komplett — vom öffnenden `<article>` bis zum schließenden `</article>`.
4. Strg + C (Kopieren), dann Pfeiltaste nach oben (Cursor an den Anfang des Blocks), dann Strg + V (Einfügen).
5. Jetzt hast du die Meldung doppelt. In der oberen Kopie änderst du:
 - Das Datum (z. B. „Oktober 2026 · Visitationsreise“).
 - Die Überschrift zwischen `<h2>` und `</h2>`.
 - Den Text zwischen `<p>` und `</p>`.
 - Die ID hinter `id=" . . . "` (vergib einen passenden Namen, z. B. `visitation-2026`).
1. Speichern (Strg + S). Browser → F5.
2. Wenn du auch eine englische Version willst: dasselbe in `en/news.html`.

Aufgabe D · Eine ganz neue Seite anlegen

Beispiel: Du willst einen Jahresbericht 2024 als eigene Seite.

1. Im Datei-Explorer den Kusaidia-Ordner öffnen.
2. Suche eine einfache Vorlage — z. B. `impressum.html`.
3. Rechtsklick → „Kopieren“.
4. Rechtsklick im selben Ordner → „Einfügen“.
5. Eine Datei `impressum - Kopie.html` entsteht. Umbenennen zu `jahresbericht-2024.html`.
6. Öffnen in VS Code.
7. Oben den `<title>` ändern.
8. Die `<h1>`-Überschrift anpassen.
9. Im `<main>`-Bereich deinen Inhalt schreiben.
10. Speichern.
11. **Wichtig — verlinken:** Damit Besucher die Seite finden, brauchst du einen Link von woanders. Z. B. in `aktuelles.html` einfügen: `<a href="jahresbericht-2024.html">Jahresbericht 2024</a>`.

Dateinamen-Regel: Nur Kleinbuchstaben, Bindestriche, keine Umlaute, kein Leerzeichen. Browser können sonst Probleme machen. Gut: `jahresbericht-2024.html` · Schlecht: `Jahresbericht 2024.html`

Aufgabe E · Einen Link einfügen

Die Grundformel ist immer:

```
<a href="WOHIN">SICHTBARER TEXT</a>
```

Drei Spielarten:

- **Auf eine andere Seite deiner Website:**

```
<a href="kontakt.html">Kontakt</a>
```

- **Auf eine externe Seite:**

```
<a href="https://www.tagesschau.de">Tagesschau</a>
```

- **Auf eine E-Mail-Adresse:**

```
<a href="mailto:info@beispiel.de">Schreib uns</a>
```

- **Auf eine Telefonnummer:**

```
<a href="tel:+4971423237">07142 32370</a>
```

Aufgabe F · Einen Spendenbetrag anpassen

Beispiel: Der Mitgliedsbeitrag soll von 40 € auf 45 € steigen.

1. `unterstuetzen.html` öffnen.
2. Strg + F (Suchen in dieser Datei).
3. Suche nach Einzelmitgliedschaft.
4. Du landest in einer Kachel `<div class="amount">`. Darin steht `<div class="amount__sum">40</div>`.
5. Ändere die 40 zu 45.
6. Strg + S. Browser → F5.
7. Englische Version: dasselbe in `en/support.html`.

Aufgabe G · Ein Vorstandsmitglied ändern

Beispiel: Es gibt einen neuen Vorstand „Maria Müller, Schriftführerin“.

1. `ueber-uns.html` öffnen.
2. Suche einen bestehenden Vorstandsblock — etwa für Dr. Michael Lutz-Dettinger.
3. Markiere den ganzen `<div class="card">...</div>`-Block.
4. Kopiere ihn und füge ihn darunter ein.
5. Im neuen Block änderst du:
 - Die Rolle nach `<span class="card__kicker">` (z. B. „Schriftführung“).

- Den Namen in `<h3>` (z. B. „Maria Müller“).
 - Die Beschreibung in `<p>`.
1. Speichern, im Browser prüfen.
 2. Wiederhole für `en/about.html`.

Aufgabe H · Ein Bild zur Galerie hinzufügen

Beispiel: Du hast ein neues Foto aus Mbulu, das in die Galerie soll.

1. Bild nach `assets/images/` legen (kleinbuchstaben-name ohne Umlaute).
2. `bildergalerie.html` öffnen.
3. Suche die passende Sektion — etwa „Schulen & Ausbildung“.
4. Innerhalb der Sektion: einen bestehenden Block `<button class="gallery-tile">...</button>` kopieren und darunter einfügen.
5. Im neuen Block den Bildpfad und die Beschriftung anpassen.
6. Zähler oben anpassen: `<span class="gallery-section__count">` — die Zahl der Aufnahmen erhöhen.
7. Speichern.
8. Auch in `en/gallery.html` nachziehen.

DEIN WICHTIGSTES SICHERHEITSNETZ

Teil 6 Git — der Schutzschirm

Git ist eine kleine Software, die jeden Arbeitsstand für dich speichert. Du kannst jederzeit zu jedem alten Stand zurück. Das nimmt dir die Angst, etwas zu verändern.

Kapitel 15 · Was ist Git?

Stell dir Git wie ein **Tagebuch** vor. Jedes Mal, wenn du etwas änderst, sagst du Git: „Bitte speichere das, was ich gerade gemacht habe.“ Git macht eine Notiz und merkt sich alle Dateien in genau diesem Zustand.

Diese Notizen heißen **Commits**. In deinem Tagebuch stehen aktuell schon ein paar Einträge:

```
6c10900 Fix gallery captions and use real Fr. Karama portrait
7a3b73a Fix callout readability
07258fb Add English version with DE/EN language switcher
d42ae24 Add photo gallery with lightbox
3c36678 Initial site rebuild from web archive content
```

Die linke Spalte ist eine eindeutige ID (ein *Hash*), die rechte ist die Notiz dazu. Wenn etwas später nicht stimmt, kannst du an jeden Punkt zurück.

Kapitel 16 · Die drei wichtigsten Befehle

Du brauchst nur drei Befehle. Sie werden im **Terminal** eingegeben.

So öffnest du das Terminal

- In VS Code: Menü **Terminal** → **Neues Terminal** (oder Strg + ö).
- Unten erscheint ein schwarzes Fenster mit Eingabeprompt.

Befehl 1 — Was hat sich geändert?

```
git status
```

Zeigt dir alle Dateien, die du seit dem letzten Commit verändert hast. Eine geänderte Datei sieht rot aus, eine markierte grün.

Befehl 2 — Alle Änderungen für den Commit markieren

```
git add .
```

Der Punkt am Ende bedeutet: „alle Dateien“. Damit teilst du Git mit: „Ich will all das gleich speichern.“

Befehl 3 — Den Commit mit einer Notiz machen

```
git commit -m "Neue Aktuelles-Meldung Oktober 2026"
```

Das `-m` heißt „message“ (Nachricht). Schreibe eine kurze Beschreibung in Anführungszeichen — Stichworte reichen.

Eselsbrücke: status → add → commit. „Schau, was ist? Alles dazu. Speichern.“

Kapitel 17 · Wenn etwas kaputt geht

Du hast noch nicht commit-et?

Dann kannst du alles auf den letzten gespeicherten Stand zurücksetzen:

```
git checkout -- .
```

Achtung: Das wirft ALLE Änderungen seit dem letzten Commit weg. Nur ausführen, wenn du sicher bist, dass die aktuelle Arbeit nicht wertvoll ist.

Du hast schon commit-et, willst aber zurück?

Zeig dir erst die letzten Commits:

```
git log --oneline
```

Dann erzeuge einen Commit, der den letzten rückgängig macht:

```
git revert HEAD
```

HEAD ist Git-Sprech für „der letzte Commit“. Du kannst auch eine spezifische ID nehmen, z. B. `git revert 6c10900`.

WAS TUN, WENN DIE SEITE HAKT?

Teil 7 Wenn etwas nicht funktioniert

Atme tief durch. Du hast nichts kaputtgemacht — Git hat alles gesichert. Hier sind die häufigsten Fehler und ihre Lösungen.

„Die Seite zeigt noch das alte Bild / den alten Text“

Der Browser merkt sich Inhalte (das nennt sich *Cache*). Drück **Strg + F5** — das lädt die Seite komplett neu.

„Ich sehe meine Änderung nicht“

- 1) Hast du die Datei gespeichert? **Strg + S**.
- 2) Hast du die richtige Datei geändert? Bist du im richtigen Ordner?
- 3) Browser mit **Strg + F5** neu laden.

„Plötzlich sieht alles komisch aus“

Wahrscheinlich ein verloren gegangenes `<` oder `>`. Bitte den Enkel um Hilfe, ODER setze die Änderungen zurück: `git checkout -- <dateiname.html>`

„Ein Bild wird nicht angezeigt“

- 1) Tippfehler im Dateinamen? Groß/Klein beachten — die Website unterscheidet `Foto.jpg` von `foto.jpg`!
- 2) Liegt das Bild wirklich in `assets/images/`?
- 3) Hat das Bild eine ungewöhnliche Endung (`.jpeg` statt `.jpg`)?

„Ein Link führt ins Leere“

Schau im `href="..."` nach. Pfad richtig geschrieben? Bei Links innerhalb von Ordnern: `projekte/sanu-tec.html`. Bei einer Datei zurück nach oben: `../index.html`.

„Eine englische Seite findet nicht zurück nach Deutsch“

Das DE/EN-Umschalter-Link auf der englischen Seite zeigt nach `../<deutsche-datei>.html`. Prüfe ihn dort.

Die Browser-Konsole — dein bester Freund

Drücke **F12** im Browser. Es öffnet sich ein technisches Fenster. Klicke auf den Reiter **Konsole** (oder *Console*).

Hier siehst du rot markierte Fehlermeldungen. Auch wenn du sie nicht alle verstehst — sie helfen einem Helfer (deinem Enkel), den Fehler zu finden. Mach im Zweifel einen Screenshot.

Allerletzte Rettung: Im Terminal `git reset --hard HEAD` eingeben. Damit gehen **alle ungespeicherten Änderungen** verloren — und die Website ist wieder auf dem letzten Commit. Das funktioniert IMMER.

WAS BEDEUTET DAS ALLES?

Anhang A Glossar

Attribut	Zusätzliche Information in einem Tag, in der Form <code>Name="Wert"</code> . Beispiel: <code>href="kontakt.html"</code> .
Browser	Programm, mit dem du das Internet ansiehst — Chrome, Firefox, Edge.
Cache	Zwischenspeicher des Browsers. Behält alte Inhalte, damit's schneller geht. Manchmal nervt das. Lösung: Strg + F5.
Class / Klasse	Ein „Etikett“ an einem HTML-Element. Das CSS schaut auf Klassen, um zu entscheiden wie etwas aussehen soll.
Commit	Ein gespeicherter Stand in Git — wie eine Notiz im Tagebuch.
CSS	<i>Cascading Style Sheets</i> . Sprache, die das Aussehen bestimmt.
Editor	Programm zum Bearbeiten von Code-Dateien. Wir nutzen VS Code.
F5	Browser-Taste: Seite neu laden.
Git	Versionsverwaltung. Speichert deine Arbeit Schritt für Schritt.
Hash	Die siebenstellige ID eines Commits — z. B. 6c10900.
HTML	<i>HyperText Markup Language</i> . Sprache des Inhalts und der Struktur.
JavaScript	Sprache des Verhaltens. Sorgt für Diashow, Menü auf/zu, klickbare Sachen.
Lokal	Auf deinem eigenen Computer (anders als „online“, auf dem Server).
Repository (Repo)	Der Ordner, der von Git verwaltet wird — bei uns also der Kusaidia-Ordner.
Server	Computer, der die Website im Internet bereitstellt.
Strg + F	In der aktuellen Datei suchen.
Strg + Shift + F	In ALLEN Dateien suchen.
Tag	HTML-Element wie <code><p></code> oder <code><h1></code> .
Terminal	Schwarzes Fenster für Befehle. In VS Code: Strg + ö.
URL	Internetadresse, z. B. kusaidia-afrika.de.

ZUM AUSDRUCKEN & NEBEN DEN RECHNER LEGEN

Anhang B Spickzettel der häufigsten Tags

Die wichtigsten HTML-Tags und Attribute auf einer Doppelseite — kompakt.

Tag	Was es macht	Beispiel
<code><h1></code>	Hauptüberschrift	<code><h1>Titel</h1></code>
<code><h2></code>	Abschnittsüberschrift	<code><h2>Abschnitt</h2></code>
<code><h3></code>	Unterüberschrift	<code><h3>Detail</h3></code>
<code><p></code>	Absatz	<code><p>Text...</p></code>
<code></code>	fett	<code>wichtig</code>
<code></code>	kursiv	<code>betont</code>
<code>
</code>	Zeilenumbruch	Zeile Zeile
<code><hr></code>	horizontaler Strich	<code><hr></code>
<code></code>	Link	<code>Klick</code>
<code></code>	Bild	<code></code>
<code></code>	Liste (ungeordnet)	<code>..</code>
<code></code>	Liste (durchnummeriert)	<code>..</code>
<code></code>	Listeneintrag	<code>Punkt</code>
<code><div></code>	Container	<code><div>...</div></code>
<code><section></code>	Abschnitt	<code><section>...</section></code>

Wichtige Attribute

Attribut	Wofür	Bei welchem Tag
href	Ziel eines Links	<code><a></code>
src	Pfad zum Bild	<code></code>

alt	Beschreibung des Bildes	
class	CSS-Klasse (Aussehen)	alle
id	Eindeutiger Name (für Anker)	alle
lang	Sprache	<html>

Tastenkürzel

Tasten	Was sie tun
Strg + S	Datei speichern
Strg + Z	Letzte Änderung rückgängig
Strg + F	In dieser Datei suchen
Strg + Shift + F	In allen Dateien suchen
Strg + C / V	Kopieren / einfügen
Strg + ö	VS Code: Terminal öffnen
F5	Browser: neu laden
Strg + F5	Browser: ganz neu laden (Cache ignorieren)
F12	Browser: Entwicklerwerkzeuge öffnen

WO FINDE ICH WAS?

Anhang C Dateibaum deiner Website

Ein Überblick: welche Datei wozu ist und wo welcher Inhalt steht.

Datei	Was drinsteht
index.html	Startseite — Hero-Slideshow, Mission, Projekt-Karten, Stats.
ueber-uns.html	Vereinsgeschichte (Margit Schnees Reise 1996), Vorstand.
projekte.html	Übersicht aller Projekte; verlinkt auf Detailseiten.
projekte/laborantenkolleg-bashanet.html	BNHCHS — Laboranten-Kolleg.
projekte/clinical-medicine.html	Clinical-Medicine-Studiengang.
projekte/sanu-tec.html	Sanu Tec Sekundarschule, Speisesaal.
projekte/madunga-girls-school.html	Madunga Girls School — Einfriedung.
projekte/student-support.html	Stipendienprogramm. Salome-Doriya-Anekdote.
partner.html	Bischof Antony, Sr. Basilisa, Fr. Karama, Diözese Mbulu.
aktuelles.html	Neuigkeiten — Kunstauktion 2024, Clinical Medicine.
unterstuetzen.html	Spendenkonto, IBAN, Mitgliedschaft, Beträge.
ihre-spende-kommt-an.html	Transparenz — wie Spenden ankommen.
bildergalerie.html	Galerie nach Themen (Land, Gesundheit, Schulen, ...).
kontakt.html	Kontaktformular, Anschrift, Rolfs Daten.
impressum.html	Pflichtangaben (Anbieter, Registernummer).
datenschutz.html	DSGVO-Erklärung.
en/	Englische Versionen — gleiche Inhalte, anderer Sprache.
css/style.css	Alle Stilangaben — Farben, Schriften, Layout.
js/main.js	Diashow, Menü auf/zu, Lightbox in der Galerie.
assets/images/	Alle Fotos. Hier kommen neue Bilder rein.

Und falls etwas hängenbleibt — ruf an. Lieber einmal zu oft.

Viel Freude beim Schrauben.

— Thomas